

Lab 07: Security Design

Lab 07: Security Design

Lab summary

These labs teach layered security in Power BI using synthetic user and territory mapping data.

Azure Government readiness

Static and dynamic RLS labs are **Gov-ready**. OLS, sensitivity labels, Purview integration, external sharing, and B2B behavior are **Verify for Gov**.

Power BI project format

Build report and semantic model artifacts as PBIP projects. PBIP is the source-controlled format for this workshop. PBIX files can be generated from PBIP later if needed.

Lab data

File	Description
https://raw.githubusercontent.com/Coding-Forge/PBI-Advanced-Factory/main/data/security/security-user-territory.csv	Synthetic dynamic RLS user-to-territory mapping.
https://raw.githubusercontent.com/Coding-Forge/PBI-Advanced-Factory/main/data/security/security-role-matrix.csv	Expected access by persona and role.

Use **Get data > Web** in Power BI Desktop to load each CSV from the raw GitHub URL.

Novice-friendly how-to guide

Create a static RLS role

1. Select **Modeling > Manage roles**.
2. Select **New**.
3. Name the role `East Region`.
4. Select `DimTerritory`.
5. Enter a filter such as `[TerritoryRegion] = "East"`.
6. Save the role.
7. Select **Modeling > View as** to test it.

Create a dynamic RLS role

1. Load `security-user-territory.csv`.
2. Relate the security table to `DimTerritory` using `TerritoryKey`.
3. Select **Modeling > Manage roles**.
4. Create a role named `Dynamic Territory Security`.
5. Filter the security table with `[UserPrincipalName] = USERPRINCIPALNAME()`.
6. Use **View as** with sample UPN values from the mapping table.

Assign Service roles safely

1. Publish only to an approved training workspace.
2. Assign users or groups to RLS roles rather than relying on report access alone.
3. Check workspace role implications before testing, because broad workspace permissions can affect what a user can do with content.
4. Use App distribution for consumers when available instead of granting broad workspace access.

Exercise 1: Static RLS

Objective: Create a role that filters data to one territory or region.

Tasks

1. Open the PBIP model from earlier modules.
2. Create a role named `East Region`.
3. Add a filter on the territory dimension for the East region.
4. Test the role in Desktop.
5. Document expected visible customers and sales.

Expected result

The role limits data to the selected region.

Exercise 2: Dynamic RLS

Objective: Create user-specific access using a security mapping table.

Tasks

1. Import `security-user-territory.csv` from its raw GitHub URL using the Web connector.
2. Relate the security table to `DimTerritory` using `TerritoryKey`.
3. Create a role named `Dynamic Territory Security`.
4. Add this filter to the security table:

```
[UserPrincipalName] = USERPRINCIPALNAME()
```

5. Test the role with sample UPN values from the mapping table.
6. Document results for users with one territory and users with multiple territories.

Expected result

The model filters territories based on the current user's mapping rows.

Exercise 3: Testing roles in Desktop and Service

Objective: Validate security before production.

Concept note: Service testing includes both data access and content distribution. Assign users or groups to roles, confirm the viewer path through the workspace or App, and record any workspace role that could change the test result.

Tasks

1. Use **View as** in Desktop.
2. Test a static role.
3. Test dynamic role with sample UPN values.
4. Publish to Service where available.
5. Assign users or groups to roles.
6. Test as role in Service.

Expected result

RLS behavior is verified in both Desktop and Service where available.

Exercise 4: Build permission behavior

Objective: Understand downstream semantic model access.

Tasks

1. Review what Build permission allows.
2. Identify which personas should have Build permission.
3. Discuss Analyze in Excel and thin report scenarios.
4. Document risks and governance controls.
5. Demonstrate in Service where available.

Expected result

Learners can distinguish report viewing from semantic model reuse.

Concept note: Sharing and external users

Direct sharing, App access, workspace access, and B2B guest access are separate decisions. External sharing and guest-user behavior are **Verify for Gov** and should be validated against tenant policy before being used as a hands-on requirement.

Optional lab: Object-level security

Azure Government note: OLS is **Verify for Gov**. Validate XMLA-compatible tooling, capacity, tenant settings, and customer policy before making this hands-on.

Objective: Understand when to hide sensitive tables or columns.

Conceptual tasks

1. Identify sensitive fields.
2. Determine whether fields should be removed, hidden, or protected with OLS.
3. Review tooling and capacity requirements.
4. Document validation requirements.

Optional lab: Sensitivity labels

Azure Government note: Sensitivity labels are **Verify for Gov**. Validate Microsoft Purview Information Protection configuration, label policies, tenant settings, and cloud support.

Objective: Understand label-based governance.

Conceptual or hands-on tasks

1. Review available labels.
2. Apply a label where available.
3. Review export and inheritance behavior.
4. Document Purview and DLP dependencies.

Validation checklist

- ☐ Static RLS role filters expected data.
- ☐ Dynamic RLS role uses `USERPRINCIPALNAME()`.
- ☐ Security mapping table has correct relationship path.
- ☐ Desktop role testing is documented.
- ☐ Service role testing is documented where available.
- ☐ Build permission behavior is documented.
- ☐ OLS is marked **Verify for Gov.**
- ☐ Sensitivity labels and Purview are marked **Verify for Gov.**
- ☐ External sharing and B2B limitations are documented.
- ☐ Security review checklist is completed.